



Architettura degli elaboratori

Lezione 7 - teoria

Prof.ssa Valentina Ciriani

Università degli Studi di Milano

www.di.unimi.it/ciriani

1



Contenuto

1. Il calcolatore, introduzione [PH 1]
2. Assembler MIPS [PH 2.1, 2.2, 2.3, 2.6]
 1. Linguaggio Assembler, introduzione [PH 2.1]
 2. Operazioni aritmetiche [PH 2.2]
 3. Operandi (registri e memoria) [PH 2.3]
 4. Operazioni logiche [PH 2.6]
 5. Moltiplicazione e divisione [PH Appendice]

-MKM = M. Morris Mano, C.R. Kime, T. Martin, Reti logiche, Pearson

-PH = D.A. Patterson, J.L. Hennessy, Struttura e Progetto dei Calcolatori, Zanichelli

2

2



Classi di calcolatori

1. **Personal computer (PC)**
 - buone prestazioni per un singolo utente
 - mantenendo il costo limitato
 - solitamente utilizzati per eseguire software scritto da terze parti
2. **Server**
 - calcolatori di grandi dimensioni
 - spesso si accede solo attraverso la rete
 - sw di terze parti o ad hoc
3. **Calcolatori embedded**
 - dedicati (aerei, automobili, televisori...)
 - sono i più numerosi
 - spesso prestazioni limitate
 - bassa dissipazione di energia
 - basso costo

3

3



Classi di calcolatori

4. **Dispositivi mobili (PMD, Personal Mobile Devices)**
 - Stanno sostituendo i PC
 - Alimentati a batteria
 - Connessione wireless
 - Basso costo
5. **Cloud computing**
 - In sostituzione dei server
 - Enormi centri di calcolo (WSC, Warehouse Scale Computers)
 - Molti server che forniscono servizi tramite internet
 - Spesso i provider noleggiano i server o la capacità di calcolo

4

4



Programmi

Un'applicazione

- è formata da milioni di linee di codice
- scritte in un linguaggio vicino a quello dell'essere umano (C, Java, Go)
- si serve di sofisticate librerie software che implementano funzioni complesse

Il **calcolatore** può eseguire:

- solo istruzioni di basso livello
- estremamente semplici

Passare da un'applicazione alle istruzioni a basso livello è un processo che utilizza vari strati di software

- che interpretano e traducono le operazioni da alto a basso livello

5

5



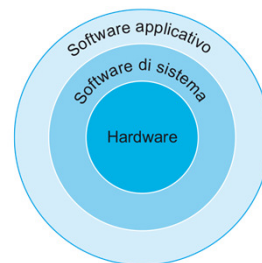
Strati di software

1. Software applicativo

- Componenti più esterne

2. Software di sistema, due componenti essenziali:

- Sistema operativo
 - Gestione I/O
 - Allocazione spazio di memoria e di disco
 - Gestione delle applicazioni che girano contemporaneamente
- Compilatori
 - Traduzione di un programma ad alto livello in istruzioni eseguibili dall'hardware



6

6



Linguaggio macchina

il **linguaggio macchina** è una composizione di numeri binari

- Un'istruzione è una stringa di bit comprensibile dal calcolatore
- Per esempio, la sequenza di bit:
00000000100010000100000000100000
è la somma di due numeri (in una particolare macchina)

7

7



Assembler

- L'assemblatore traduce la versione simbolica delle istruzioni nella corrispondente forma binaria
- Ad ogni istruzione Assembler corrisponde esattamente una istruzione in linguaggio macchina
- Per esempio, un programmatore potrebbe scrivere:
 - Assembler: `add C, A, B`
 - Linguaggio macchina (traduzione)
00000000100010000100000000100000
 - comando che calcola la somma dei due numeri A e B e la mette in C

8

8



Linguaggi ad alto livello

- Sono linguaggi più vicini al pensiero umano e più efficienti da scrivere
 - Linguaggio ad alto livello: $C = A + B$
 - Assembler: `add C, A, B`
 - Linguaggio macchina:
00000000100010000100000000100000
- Ogni istruzione in linguaggio ad alto livello può corrispondere a **più istruzioni** Assembler

9

9



Esempio

Programma
scritto
in linguaggio
ad alto livello (in C)

```
Scambia(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

↓
Compilatore

Programma
scritto
in linguaggio
assembler
(per il MIPS)

```
Scambia:
  multi $9, $5, $2
  add $9, $4, $9
  lw $15, 0($9)
  lw $16, 4($9)
  sw $16, 0($9)
  sw $15, 4($9)
  jr $31
```

↓
Assemblatore

Programma
scritto
in linguaggio
macchina
(del MIPS)

```
00000000000001010100100010000000
00000001001001001000100000100000
10001101001011110000000000000000
10001101001100000000000000000000
10101101001011110000000000000000
101011010011000000000000000100
000001111100000000000000001000
```

10

10



Architettura di Von Neumann

Componenti principali:

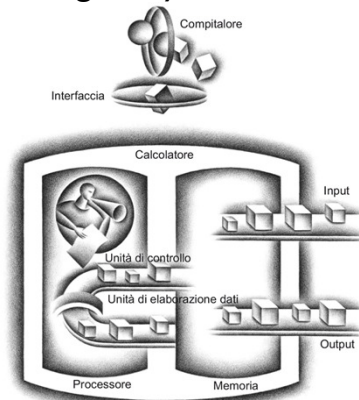
- **Processore (CPU, Central Processing Unit)**

- Unità di elaborazione dati (datapath)
- Unità di controllo

- **Memoria principale** per memorizzare istruzioni e dati

- **Dispositivi di input** per l'acquisizione dati dall'esterno

- **Dispositivi di output** per fornire dati all'esterno



11

11



Architettura di Von Neumann

Caratteristiche principali:

- Un programma per essere eseguito si deve trovare in memoria principale
- La memoria contiene sia dati che istruzioni
- La memoria è leggibile e scrivibile
- Le istruzioni di un programma sono eseguite in modo sequenziale

12

12



Processore

Ha due componenti:

1. **unità di elaborazione dati** (datapath)
 1. ALU (unità aritmetico-logica)
 - esegue le operazioni aritmetiche e logiche
 2. registri
 - la memoria locale al processore
2. **unità di controllo**
 - indica al datapath, alla memoria e ai dispositivi di I/O che cosa fare a seconda di quanto specificato nelle istruzioni di un programma

13

13



Memoria principale

La memoria, che può essere letta e scritta, contiene:

- i **programmi** quando vengono eseguiti
 - i **dati** richiesti dalla loro esecuzione
- E' fatta con la tecnologia **DRAM** (memoria dinamica ad accesso casuale):
- costruita con circuiti integrati
 - richiede lo stesso tempo di accesso in qualsiasi posizione della memoria (**accesso casuale**)
 - è **volatile**
 - i bit devono essere ricaricati periodicamente (**dinamica**)
 - piuttosto veloce (**media velocità**)
 - non molto costosa (**medio costo**)

14

14



Memoria principale

- La memoria si può astrarre come un **vettore di celle** (o locazioni) di memoria
- Ogni cella è costituita da un numero prefissato di bit: la **dimensione** (es: 8 bit, 32 bit)
- Ogni cella è identificata da un intero senza segno: l'**indirizzo di memoria**
- L'**accesso** ad una locazione (per operazioni di lettura e scrittura) avviene specificando l'indirizzo

Indirizzi	Dimensione: 8 bit
149	
150	1 0 0 0 1 1 1 1
151	0 0 0 0 1 0 0 0
152	1 0 0 0 1 0 1 0
153	0 0 0 1 0 0 1 1
154	0 0 0 1 0 1 1 0
155	
...	
162	

15

162

15



Ciclo fetch-execute

- Le istruzioni del programma in esecuzione
 - sono in linguaggio macchina
 - risiedono in memoria principale
 - sono disposte in sequenza nell'ordine in cui sono scritte dal programmatore

Il processore preleva un'istruzione alla volta e la esegue con il ciclo **fetch-execute**:

- Fetch**: preleva l'istruzione dalla memoria centrale
- Execute**: decodifica l'istruzione e la esegue

16

16



Altre memorie

- Oltre ai registri (nella CPU) e alla memoria principale, esistono altri tipi di memoria
- Guardiamone i principali e come sono messi in gerarchia

17

17



Memoria cache

Piccola memoria all'interno del processore, che può essere letta e scritta, contiene:

- istruzioni e dati **usati di recente**

E' fatta con la tecnologia **SRAM** (memoria statica ad accesso casuale):

- è ad **accesso casuale**
- è **volatile**
- i bit non hanno bisogno di essere riscritti (**statica**)
- **molto veloce**
- **molto costosa** perché meno densa

18

18



Memoria secondaria

La memoria secondaria, che può essere letta e scritta, contiene:

- i programmi
- i dati

E' fatta con diverse tecnologie (dischi magnetici, memorie flash):

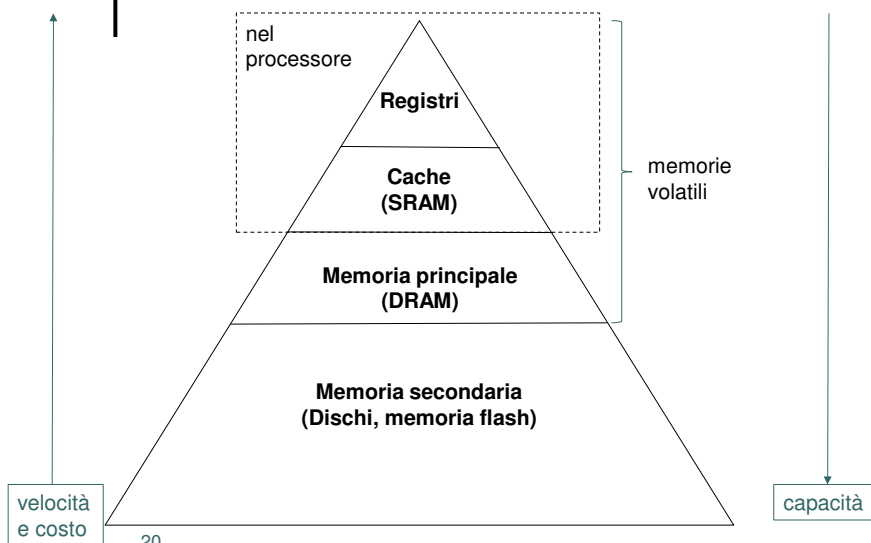
- **non è volatile** (mantiene i dati anche in mancanza di alimentazione)
- **i tempi di accesso variano** a secondo della posizione in memoria
- **molto lenta**
- **poco costosa**

19

19



Gerarchia di memoria



20

20



Architettura

L'architettura dell'insieme di istruzioni (ISA, Instruction Set Architecture)

- è l'interfaccia astratta tra l'hardware e il livello più basso del software di un calcolatore
- comprende
 - tutte le informazioni necessarie per scrivere un programma in linguaggio macchina funzionante
 - le istruzioni e il loro formato binario
 - i registri nel processore
 - le caratteristiche della memoria principale
 - i dispositivi di I/O

21

21



Implementazione

- è l'hardware che realizza la descrizione astratta dell'architettura
- ci possono essere processori diversi che si basano sulla stessa ISA ma forniscono implementazioni differenti

22

22



Classificazione architetture

CISC Complex Instruction Set Computer:

- Elevato numero di istruzioni disponibili
- Istruzioni con molti formati differenti
- Istruzioni complesse
- Elevata complessità della CPU

RISC Reduced Instruction Set Computer:

- Ridotto numero di istruzioni disponibili
- Istruzioni con pochi formati differenti
- Istruzioni semplici
- CPU semplice
- tempi ridotti di esecuzione delle singole istruzioni
- ottimizzazione delle istruzioni semplici (che sono, quasi sempre, quelle più utilizzate)

23

23



Architettura MIPS

L'architettura MIPS (Microprocessor without Interlocked Pipeline Stages)

- è un'architettura RISC
- è quella che useremo d'ora in poi

24

24



Operazioni aritmetiche

`add a,b,c`

- la somma di `b` e `c` è messa in `a`
- notazione rigida
- ciascuna istruzione aritmetica MIPS
 - esegue solo un'operazione
 - deve contenere esattamente tre variabili
- questo rende l'hardware semplice
- Esempio: `a` deve contenere la somma di `b`, `c`, `d` ed `e`

```
add a,b,c    # la somma di b e c è posta in a
add a,a,d    # la somma di b, c e d è ora posta in a
add a,a,e    # la somma di b, c, d ed e è ora posta in a
```

25

25



Esempio

- Frammento di programma in un linguaggio ad alto livello

```
a = b + c
d = a - e
```

- Assembler MIPS (prima approssimazione):

```
add a,b,c
sub d,a,e
```

26

26



Esempio

- Frammento di programma in un linguaggio ad alto livello

```
f = (g + h) - (i + j)
```

- Assembler MIPS (prima approssimazione):

```
add t0,g,h      # la variabile temporanea t0
                 # contiene g + h
add t1,i,j      # la variabile temporanea t1
                 # contiene i + j
sub f,t0,t1     # f assume il valore t0-t1,
                 # ovvero (g+h)-(i+j)
```

27

27



Registri

- devono essere scelte tra un numero limitato di locazioni chiamate **registri**
- Nell'architettura MIPS i registri hanno dimensione di 32 bit (1 parola, word)
- Il numero di registri è 32
- Convenzione:
 - variabili dei programmi: \$s0, \$s1, ..., \$s7
 - registri temporanei: \$t0, \$t1, ..., \$t9
 - il registro \$zero contiene sempre il valore 0

28

28



Esempio (con registri)

- Frammento di programma in un linguaggio ad alto livello

```
f = (g + h) - (i + j)
```

- Assembler MIPS (corretto):

```
add $t0,$s1,$s2 # il registro temporaneo
                  # $t0 contiene il risultato g + h
add $t1,$s3,$s4 # il registro temporaneo
                  # $t1 contiene il risultato i + j
sub $s0,$t0,$t1 # f assume il valore $t0 - $t1,
                  # cioè (g + h) - (i + j)
```

29

29



Operandi immediati

- Spesso si fanno operazioni con costanti
- Le istruzioni viste fino ad ora usano come operandi i registri
- Per fare operazioni con costanti usiamo le istruzioni "immediate"

Esempio:

```
addi $s3,$s3,4    # $s3 = $s3 + 4
```

30

30



Copia di registri

L'operazione di copiatura dati tra due registri

- è un'istruzione di somma nella quale uno dei due addendi è lo zero

```
addi $s1,$s2,0    # $s1 = $s2
```

31

31



Sottrazione immediata

- `subi` **non esiste**
- ma basta usare `addi` con una costante negativa

```
addi $s1,$s2,-2    # $s1 := $s2 - 2
```

32

32



Array e matrici

- Come gestire strutture più complesse di semplici variabili?
- Sono in memoria
- Le istruzioni aritmetiche usano i registri
- Dobbiamo trasferire i dati dalla memoria ai registri e viceversa
- Per accedere ad una cella di memoria uso il suo indirizzo

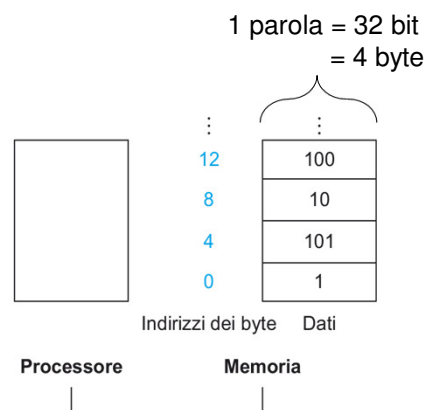
33

33



Memoria

- La **dimensione delle celle** è di 8 bit (1 byte)
- Ogni cella ha un **indirizzo** di memoria rappresentato con 32 bit
- La **dimensione massima della memoria** è quindi: 2^{32} byte (4 GigaByte)
- I byte sono **raggruppati** in parole di 4 byte (word).
- Le parole iniziano ad un indirizzo che è **multiplo di 4**, ad esempio: 0, 4, 8...



34

34

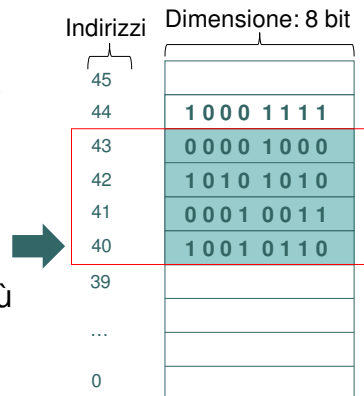


Byte in una parola

- I byte all'interno di una parola possono essere disposti secondo due criteri:

- Big-endian:** byte più significativo con indirizzo più basso
- Little-endian:** byte più significativo con indirizzo più alto

- MIPS usa big-endian



1001 0110 0001 0011 1010 1010 0000 1000

35

35



Lettura di una parola

- load in MIPS è `lw`, che sta per load word (carica una parola)
- È il trasferimento di una parola da memoria a registro

Esempio

`lw $s1, 20($s2)`

Significato

`$s1 = Memoria[$s2+20] (A[5])`

36

36



Esempio

- A vettore di 100 parole
- il compilatore associa le variabili g e h ai registri $\$s1$ e $\$s2$
- l'indirizzo base del vettore è memorizzato in $\$s3$
- Si traduca in assembler il seguente assegnamento scritto in un linguaggio ad alto livello :

$$g = h + A[8]$$

37

37



Esempio

Indirizzo
di memoria

$$g = h + A[8]$$

```
lw $t0, 32($s3)    # $t0 prende il
                   # valore di A[8]
                   # 32 = 8x4
add $s1, $s2, $t0  # g = h + A[8]
```

38

38



Scrittura di una parola

- store in MIPS è `sw`, che sta per store word (memorizza una parola)
- È il trasferimento di una parola da registro a memoria

indice del vettore A è $20/4 = 5$

indirizzo base del vettore A

Esempio

`sw $s1, 20($s2)`

Significato

`Memoria[$s2+20] = $s1`

39

39



Esempio

- A vettore di 100 parole
- il compilatore associa la variabile `h` al `$s2`
- l'indirizzo base del vettore è memorizzato in `$s3`
- Si traduca in assembler il seguente assegnamento scritto:

`A[12] = h + A[8]`

40

40



Esempio

$$A[12] = h + A[8]$$

```
lw $t0, 32($s3)    # $t0 prende il
                   # valore di A[8]
add $t0, $s2, $t0  # $t0 = h + A[8]
sw $t0, 48($s3)    # h + A[8] è
                   # memorizzato
                   # in A[12]
```

41

41



Memoria e registri MIPS

Operandi MIPS

Nome	Esempio	Commenti
32 registri	<code>\$s0-\$s7, \$t0-\$t9, \$zero,</code> <code>\$a0-\$a3, \$v0-\$v1, \$gp,</code> <code>\$fp, \$sp, \$ra, \$at</code>	Accesso veloce ai dati. Nel MIPS gli operandi devono essere contenuti nei registri per potere eseguire delle operazioni. Il registro \$zero contiene sempre il valore 0, e il registro \$at viene riservato all'assemblatore per la gestione di costanti molto lunghe.
2 ³⁰ parole di memoria	Memoria[0], Memoria[4], Memoria[4294967292]	Alla memoria si accede solamente attraverso le istruzioni di trasferimento dati. Il MIPS utilizza l'indirizzamento al byte, perciò due parole consecutive hanno indirizzi in memoria a una distanza di 4. La memoria consente di memorizzare strutture dati, vettori, o il contenuto dei registri.

42

42



Operatori logici

Operazioni logiche	Operatori C	Operatori Java	Istruzioni MIPS
Shift a sinistra	<<	<<	sll
Shift a destra	>>	>>>	srl
AND bit a bit	&	&	and, andi
OR bit a bit			or, ori
NOT bit a bit	~	~	nor

43

43



Shift left

`sll $t2, $s0, 4` # `$t2 = $s0 << 4 bit`

0000 0000 0000 0000 0000 0000 0000 1001 = 9_{10}
 0000 0000 0000 0000 0000 0000 1001 0000 = 144_{10}

`sll`, shift left logical

Sposta a sinistra di 4 bit e mette degli zeri.
 Equivale a moltiplicare per $(2^4)_{10} = 16_{10}$

44

44



Esempio

Tradurre $y = A[i]$ in assembler MIPS dove

- le variabili y e i sono nei registri $\$s1$ e $\$s2$
- l'indirizzo base di A è in $\$s0$
- Ricordarsi che gli indici sono multipli di 4 e per moltiplicare per 4 basta fare uno shift left di 2^2

lw \$s1, \$s2×4(\$s0) NO

```
sll $t0, $s2, 2    # $t0 = i × 4
add $t0, $s0, $t0  # $t0 contiene
                   # l'indirizzo di A[i]
lw $s1, 0($t0)     # y = A[i]
```

45

45



Shift right

```
srl $t2, $s0, 4    # $t2 = $s0 >> 4 bit
```

```
0000 0000 0000 0000 0000 0000 0010 1001 = 4110
```

```
0000 0000 0000 0000 0000 0000 0000 0010 = 210
```

srl, shift right logical

Nell'esempio:

- sposta a destra di 4 bit e mette degli zeri
- equivale alla divisione intera per $(2^4)_{10} = 16_{10}$

46

46



AND bit a bit

```
and $t0,$t1,$t2      # $t0 = $t1 & $t2
```

```
0000 0000 0000 0000 0000 0000 0010 1011  $t1
0000 0000 0000 0000 0000 0000 1010 0010  $t2
0000 0000 0000 0000 0000 0000 0010 0010  $t0
```

Può essere usata per:

- per forzare a 0 alcuni bit di una parola fornendo in input all'AND una parola che contiene 0 in quelle posizioni
- dato che «nasconde» alcuni bit, viene chiamata maschera

47

47



OR bit a bit

```
or $t0,$t1,$t2      # $t0 = $t1 | $t2
```

```
0000 0000 0000 0000 0000 0000 0010 1011  $t1
0000 0000 0000 0000 0000 0000 1010 0010  $t2
0000 0000 0000 0000 0000 0000 1010 1011  $t0
```

Può essere usata per:

- forzare ad 1 alcuni bit di una parola fornendo in input all'OR una parola che contiene 1 in quelle posizioni

48

48



NOR bit a bit

```
nor $t0,$t1,$t2    # $t0 = ~($t1 | $t2)
```

```
0000 0000 0000 0000 0000 0000 0010 1011  $t1
0000 0000 0000 0000 0000 0000 1010 0010  $t2
1111 1111 1111 1111 1111 1111 0101 0100  $t0
```

Può essere usata per simulare il NOT ovvero se \$t2 contiene uno 0 allora \$t0 contiene ~\$t1
infatti

- $A \text{ NOR } 0 = \text{NOT } (A \text{ OR } 0) = \text{NOT } A$
- oppure $A \text{ NOR } A = \text{NOT } (A \text{ OR } A) = \text{NOT } A$

49

49



NOT usando il NOR

le variabili x e y sono nei registri \$s1 e \$s2
inserire in x il NOT di y

Due possibili modi:

```
nor $s1,$s2,$s2    # $s1 = ~($s2|$s2)
```

```
nor $s1,$s2,$zero  # $s1 = ~($s2|$zero)
```

50

50



ANDi e ORi

```
ori $t0,$t1,0xFFFF # costante max 16 bit
```

```
1111 1011 1111 1111 0000 0000 0010 1011 $t1
0000 0000 0000 0000 1111 1111 1111 1111
1111 1011 1111 1111 1111 1111 1111 1111 $t0
```

```
andi $t0,$t1,0xFFFF0 # costante max 16 bit
```

```
0000 0000 0000 0000 0100 0000 0010 1011 $t1
0000 0000 0000 0000 1111 1111 1111 0000
0000 0000 0000 0000 0100 0000 0010 0000 $t0
```

51

51



Moltiplicazione

```
mult $s0 $s1 # $s0 * $s1
```

- Il risultato della moltiplicazione viene posto sempre in **due registri dedicati**:
 - hi (High order word)
 - lo (Low order word)
- La moltiplicazione di due numeri rappresentabili con 32 bit può dare un risultato non rappresentabile in 32 bit
- Il risultato della moltiplicazione si preleva dai registri hi e lo utilizzando le due istruzioni:

```
mfhi $t1 # move from hi to $t1
```

```
mflo $t2 # move from lo to $t2
```

52

52

Esempio

```
mult $s0 $s1 # $s0 * $s1
```

```
01111111111111111111111111111111x    $s0
```

00100000000000000000000000000000= \$s1

```
00001111111111111111111111111111|11100000000000000000000000000000
```

hi lo

mfhi \$t1

```
00001111111111111111111111111111 $t1
```

mf1o \$t2

```
11100000000000000000000000000000 $t2
```

53

● ● ● **Divisione**

```
div $s0 $s1 #divide $s0 per $s1
```

Anche il risultato della divisione viene posto sempre in **due registri dedicati**:

- hi il resto
- lo il quoziente

54



Esercizi da fare a casa 1

1. Qual è il codice assembler MIPS corrispondente alla seguente istruzione? Si supponga che le variabili f , g e h siano date e che possano essere dichiarate come interi a 32 bit all'interno del programma:

$$f = g + (h - 5)$$

2. Scrivere una sola istruzione ad alto livello che corrisponda alle seguenti istruzioni MIPS:

```
add f, g, h
add f, i, f
```

55

55



Esercizi da fare a casa 2

3. Scrivere il codice assembler MIPS che corrisponde alla seguenti istruzioni ad alto livello. Si supponga che le variabili i e j , siano assegnate rispettivamente ai registri $\$s3$ e $\$s4$. Si assuma che l'indirizzo base dei vettori A e B sia contenuto rispettivamente nei registri $\$s6$ e $\$s7$

a) $B[8] = A[i-j]$

b) $B[8] = A[i] + A[j]$

56

56



Esercizi da fare a casa 3

4. Tradurre il seguente spezzone di codice assembler MIPS in un linguaggio ad alto livello. Si supponga che le variabili *f*, *g*, *h*, *i* e *j* siano assegnate rispettivamente ai registri *\$s0*, *\$s1*, *\$s2*, *\$s3* e *\$s4*. Si assuma che l'indirizzo base dei vettori *A* e *B* sia contenuto nei registri *\$s6* e *\$s7* rispettivamente.
- ```
addi $t0, $s6, 4
add $t1, $s6, $zero
sw $t1, 0($t0)
lw $t0, 0($t0)
add $s0, $t1, $t0
```

57

57



## Esercizi da fare a casa 4

5. Supponendo che *\$t0* = 0xAAAAAAAA, *\$t1* = 0x12345678 determinare il contenuto di *\$t2* dopo l'esecuzione delle seguenti istruzioni:
- ```
sll $t2, $t0, 4
or $t2, $t2, $t1
```

58

58



Esercizi da fare a casa 5

6. Supponendo che `$t0 = 0xAAAAAAAA`,
`$t1 = 0x12345678` determinare il
 contenuto di `$t2` dopo l'esecuzione
 delle seguenti istruzioni:
`srl $t2, $t0, 3`
`andi $t2, $t2, 0xFFEF`

59

59



Esercizi da fare a casa 6

7. Determinare la sequenza più corta di
 istruzioni MIPS che consente di estrarre i bit
 da 11 a 16 dal registro `$t0` e li sostituisce
 ai bit da 26 a 31 del registro `$t1` senza
 modificare gli altri bit dei registri `$t0` e `$t1`
 (provare il vostro codice con `$t0 =`
`0xFFFE27FF` e `$t1 = 0xFFFFFFFF`).
 Attenzione le istruzioni `ori` e `andi` usano
 delle costanti con al massimo 16 bit
 (Es: `ori $t0, $t1, 0xFFFF`)

60

60



Esercizi da fare a casa 7

8. Determinare una sequenza di istruzioni MIPS che si possono utilizzare per mettere in $\$t2$ il NOT del contenuto di $\$t1$
9. Determinare una sequenza di istruzioni MIPS che si possono utilizzare per tradurre la seguente istruzione:
 $A = C[0] \ll 4$
Si supponga che $\$t1 = A$, $\$t2 = B$ e $\$s1$ contenga l'indirizzo base di C

61